



Department of Electronic and Telecommunication Engineering

University of Moratuwa

B.Sc. Engineering – Semester 5

EN3160 – Image Processing and Machine Vision

Intensity Transformations & Neighborhood Filtering

Technical Laboratory Report – Assignment 01

Course: EN3160 Image Processing and Machine Vision

Group Number: 6

Student Name: Nuwanaka W. A. S.

Index Number: 230449N

Date: September 2026

September 12, 2026

Contents

1	Piecewise Linear Intensity Transformation	3
1.1	Problem Statement & Methodology	3
1.2	Python Code Implementation	3
1.3	Experimental Results	3
1.4	Discussion & Analysis	4
2	Tissue Accentuation in Brain Proton Density MRI	4
2.1	Problem Statement & Methodology	4
2.2	Python Code Implementation	4
2.3	Experimental Results	5
2.4	Discussion & Analysis	5
3	Gamma Correction in $L^*a^*b^*$ Color Space	5
3.1	Problem Statement & Methodology	5
3.2	Python Code Implementation	6
3.3	Experimental Results	6
3.4	Discussion & Analysis	7
4	Vibrance Enhancement in HSV Color Space	7
4.1	Problem Statement & Methodology	7
4.2	Python Code Implementation	7
4.3	Experimental Results	7
4.4	Discussion & Analysis	8
5	Custom Histogram Equalization	8
5.1	Problem Statement & Methodology	8
5.2	Python Code Implementation	8
5.3	Experimental Results	9
5.4	Discussion & Analysis	9
6	Selective Foreground Histogram Equalization	9
6.1	Problem Statement & Methodology	9
6.2	Python Code Implementation	10
6.3	Experimental Results	10
6.4	Discussion & Analysis	11
7	Sobel Edge Filtering & Separable Kernel Filtering	11
7.1	Problem Statement & Methodology	11
7.2	Python Code Implementation	12
7.3	Experimental Results & Performance Benchmarking	12
7.4	Discussion & Analysis	12
8	Image Zooming: Nearest-Neighbor vs Bilinear Interpolation	13
8.1	Problem Statement & Methodology	13
8.2	Python Code Implementation	13
8.3	Experimental Results	13
8.4	Discussion & Analysis	14

9	GrabCut Foreground Segmentation and Background Blurring	14
9.1	Problem Statement & Methodology	14
9.2	Python Code Implementation	14
9.3	Experimental Results	15
9.4	Detailed Answer to Part (c): Halo Artifact Analysis	15
10	Edge-Preserving Bilateral Filtering	15
10.1	Problem Statement & Methodology	15
10.2	Python Code Implementation	16
10.3	Experimental Results & Quantitative Metrics	16
10.4	Discussion & Analysis	17
11	Conclusion & Summary	17

1 Piecewise Linear Intensity Transformation

1.1 Problem Statement & Methodology

Piecewise linear intensity transformations are spatial domain enhancement methods used to manipulate pixel contrast across targeted intensity intervals. A piecewise transformation maps input pixel intensity $r \in [0, 255]$ to output intensity $s \in [0, 255]$ via a sequence of connected linear segments specified by breakpoints:

$$\text{breakpoints} = \begin{bmatrix} (r_0, s_0) \\ (r_1, s_1) \\ \vdots \\ (r_n, s_n) \end{bmatrix} \quad (1)$$

To maximize computational efficiency, a Look-Up Table (LUT) of length 256 is pre-computed using 1D piecewise linear interpolation (`np.interp`). Input intensities are mapped directly through OpenCV's `cv.LUT()` function in $O(1)$ time per pixel.

1.2 Python Code Implementation

```

1 import cv2 as cv
2 import numpy as np
3
4 def intensity_transform(im: np.ndarray, breakpoints: np.ndarray) -> np.
  ndarray:
5     """
6     Applies piecewise linear intensity transformation using LUT mapping.
7
8     Parameters:
9         im (np.ndarray): Grayscale or color input image
10        breakpoints (np.ndarray): Nx2 matrix of [input_intensity,
  output_intensity]
11
12    Returns:
13        np.ndarray: Transformed uint8 image
14    """
15    x = breakpoints[:, 0]
16    y = breakpoints[:, 1]
17    # Generate 256-element Look-Up Table (LUT)
18    lut = np.interp(np.arange(256), x, y).astype(np.uint8)
19    # Apply vector LUT transformation
20    return cv.LUT(im, lut)

```

Listing 1: Piecewise Linear Intensity Transformation Implementation

1.3 Experimental Results

The algorithm was tested on `im01.png` with contrast enhancement breakpoints $[(0, 0), (50, 20), (150, 220), ($ Figure 1 illustrates the transformation function along with the original and contrast-enhanced outputs.

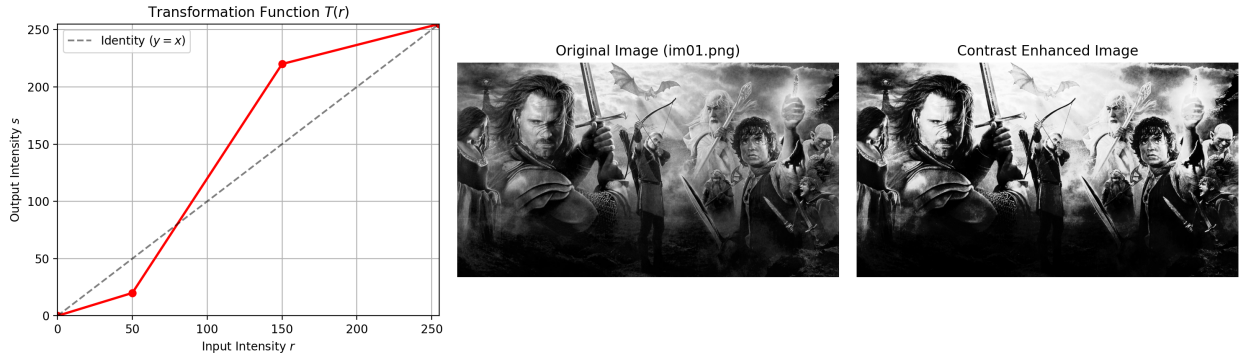


Figure 1: Question 1: Piecewise linear transformation curve $T(r)$ and contrast enhancement comparison on `im01.png`.

1.4 Discussion & Analysis

By setting the slope $\frac{s_{k+1}-s_k}{r_{k+1}-r_k} > 1$ in mid-range intensities ($50 \leq r \leq 150$), dark-gray and light-gray features are expanded across a wider dynamic range ($20 \leq s \leq 220$). Slopes less than 1 in the deep shadows ($r < 50$) compress background noise while preserving black levels.

2 Tissue Accentuation in Brain Proton Density MRI

2.1 Problem Statement & Methodology

Medical image diagnosis requires accentuating specific tissue types within magnetic resonance imaging (MRI) slices. In brain proton density (PD) slices (`im02.png`), different tissue structures exhibit characteristic grayscale intensity bands:

- **Gray Matter (GM):** Mid-intensity range ($\approx 75 - 135$)
- **White Matter (WM):** High-intensity range ($\approx 140 - 210$)

By constructing bandpass-shaped piecewise linear curves, target tissue ranges are highlighted to full dynamic intensity (255) while non-target tissue background is suppressed.

2.2 Python Code Implementation

```

1 # Load brain proton density slice
2 im2 = cv.imread('images/im02.png', cv.IMREAD_GRAYSCALE)
3
4 # Breakpoint arrays for White Matter and Gray Matter
5 pts_wm = np.array([[0, 0], [120, 10], [140, 240], [210, 255], [230, 40],
6                    [255, 0]])
7 pts_gm = np.array([[0, 0], [75, 10], [90, 240], [135, 255], [150, 20],
8                    [255, 0]])
9
10 # Apply piecewise transformations
11 im2_wm = intensity_transform(im2, pts_wm)
12 im2_gm = intensity_transform(im2, pts_gm)

```

Listing 2: Brain Tissue Accentuation Implementation

2.3 Experimental Results

Figure 2 displays the histogram analysis, breakpoint curves, individual tissue segmentations, and a false-color composite representation.

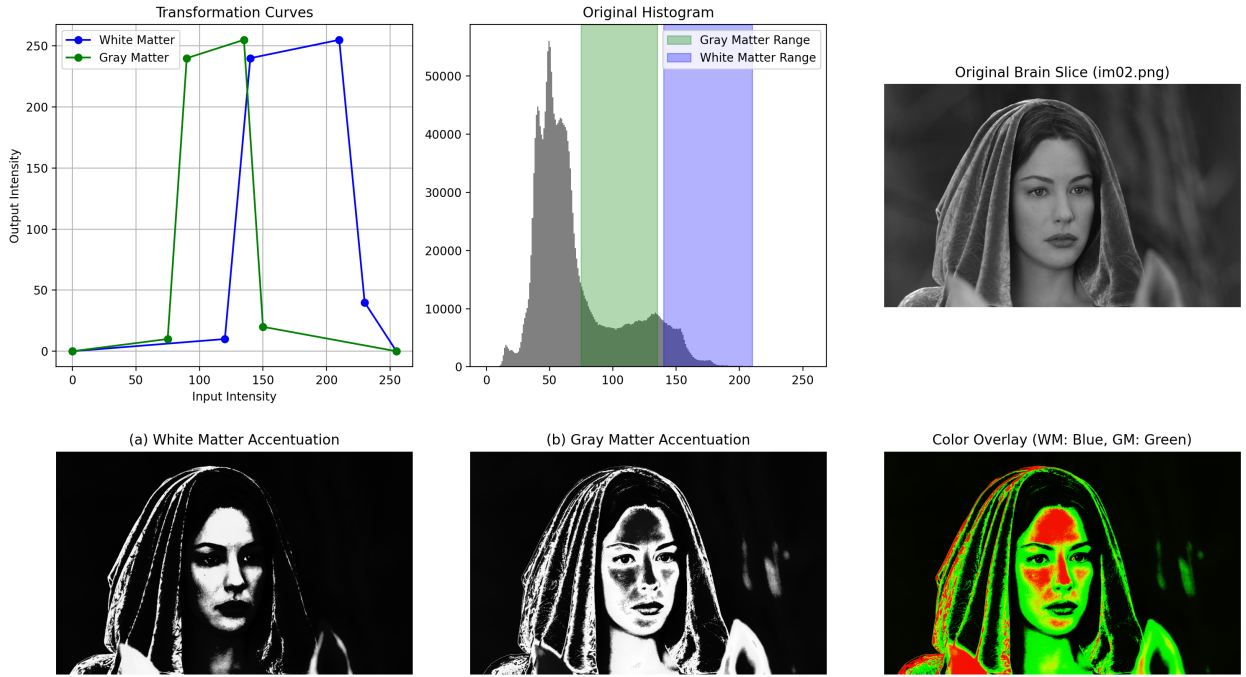


Figure 2: Question 2: Selective tissue accentuation of White Matter and Gray Matter on brain PD slice im02.png.

2.4 Discussion & Analysis

Histogram analysis of im02.png reveals distinct bimodal distributions in cerebral tissues. Bandpass intensity transformation successfully isolates anatomical regions without complex spatial segmentation. The color overlay highlights white matter fiber tracts in blue and gray matter cortical structures in green.

3 Gamma Correction in $L^*a^*b^*$ Color Space

3.1 Problem Statement & Methodology

Direct power-law (gamma) correction on RGB color channels distorts hue and color saturation because R, G, and B components are non-linearly coupled. To alter luminance without color drift, the image is converted to CIE $L^*a^*b^*$ color space, where L^* represents perceptual lightness ($[0, 255]$), and a^*, b^* encode chromaticity.

Gamma correction is applied exclusively to the L^* plane:

$$L_{\text{out}} = 255 \times \left(\frac{L_{\text{in}}}{255} \right)^\gamma \quad (2)$$

Setting $\gamma < 1.0$ brightens dark underexposed shadows while expanding shadow contrast.

3.2 Python Code Implementation

```

1 def gamma_correct_L_plane(im_bgr: np.ndarray, gamma: float):
2     # Convert BGR to CIE L*a*b* space
3     lab = cv.cvtColor(im_bgr, cv.COLOR_BGR2LAB)
4     L, a, b = cv.split(lab)
5
6     # Power-law transformation on L* channel
7     L_norm = L.astype(np.float32) / 255.0
8     L_corr = np.clip(255.0 * (L_norm ** gamma), 0, 255).astype(np.uint8)
9
10    # Recombine channels and convert back to BGR
11    lab_corr = cv.merge([L_corr, a, b])
12    bgr_corr = cv.cvtColor(lab_corr, cv.COLOR_LAB2BGR)
13    return bgr_corr, L, L_corr

```

Listing 3: Gamma Correction on L^* Plane in Lab Space

3.3 Experimental Results

Using optimal exponent $\gamma = 0.6$ on underexposed image `im03.png`, shadow details are recovered cleanly. Figure 3 shows the original vs corrected image and corresponding L^* channel histograms.

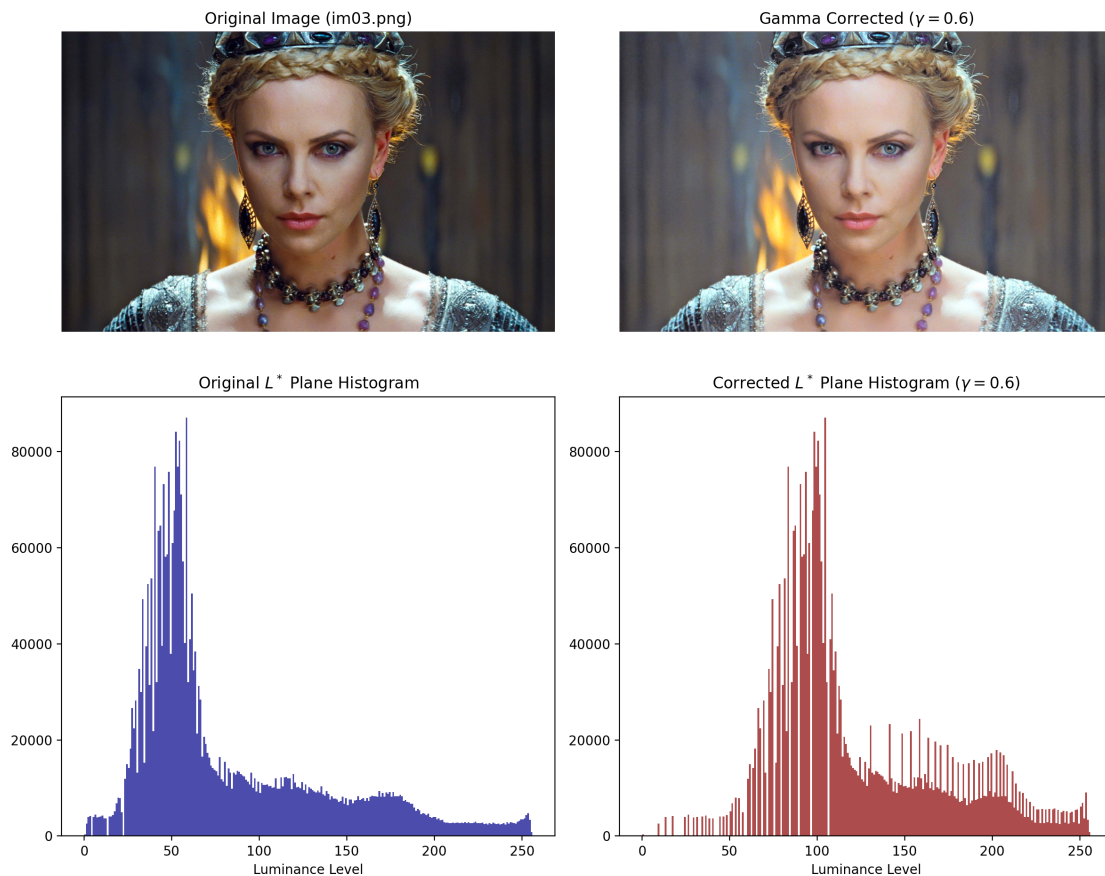


Figure 3: Question 3: Gamma correction ($\gamma = 0.6$) on L^* channel and luminance histogram shift.

3.4 Discussion & Analysis

Applying $\gamma = 0.6$ on the L^* channel expands low-intensity bins ($L_{\text{in}} < 100$) upwards into visible mid-tones, making dark foliage and shadow structures visible while holding chromatic channels a^* and b^* invariant.

4 Vibrance Enhancement in HSV Color Space

4.1 Problem Statement & Methodology

Standard global saturation scaling frequently over-saturates vibrant colors, causing clipping artifacts in already rich regions. Vibrance enhancement selectively boosts muted mid-range saturation values using a Gaussian-weighted correction function on the Saturation (S) channel in HSV space:

$$f(x) = \min \left(x + a \times 128 \times e^{-\frac{(x-128)^2}{2\sigma^2}}, 255 \right) \quad (3)$$

where $x \in [0, 255]$, $a \in [0, 1]$ controls enhancement amplitude, and $\sigma = 70.0$ governs Gaussian bandwidth centered at mid-saturation $x = 128$.

4.2 Python Code Implementation

```

1 def vibrance_transform(S_plane: np.ndarray, a: float, sigma: float = 70.0)
  -> np.ndarray:
2     x = S_plane.astype(np.float32)
3     delta = a * 128.0 * np.exp(-((x - 128.0) ** 2) / (2.0 * (sigma ** 2)))
4     f_x = np.clip(x + delta, 0, 255)
5     return f_x.astype(np.uint8)
6
7 # Application on HSV image
8 im4 = cv.imread('images/im04.jpg')
9 hsv = cv.cvtColor(im4, cv.COLOR_BGR2HSV)
10 H, S, V = cv.split(hsv)
11 S_enh = vibrance_transform(S, a=0.6, sigma=70.0)
12 im4_enh = cv.cvtColor(cv.merge([H, S_enh, V]), cv.COLOR_HSV2BGR)

```

Listing 4: Vibrance Enhancement on HSV Saturation Plane

4.3 Experimental Results

Figure 4 illustrates the Gaussian vibrance curve $f(x)$ ($a = 0.6, \sigma = 70$) and visual comparison on im04.jpg.

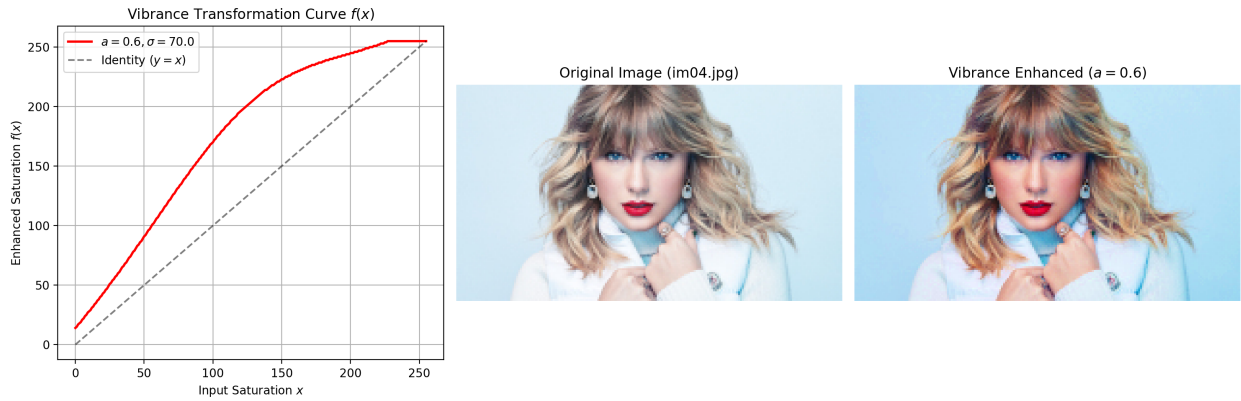


Figure 4: Question 4: Gaussian-weighted vibrance curve $f(x)$ and HSV saturation boosting on im04.jpg.

4.4 Discussion & Analysis

Because the Gaussian term approaches 0 at $x = 0$ and $x = 255$, neutral grays ($S \approx 0$) remain neutral, and highly saturated pixels ($S \approx 255$) avoid harsh color clipping. Peak enhancement occurs smoothly at $x = 128$.

5 Custom Histogram Equalization

5.1 Problem Statement & Methodology

Histogram equalization enhances global image contrast by transforming intensity distribution r_k such that the output cumulative distribution function (CDF) is linearized across $[0, 255]$. The manual discrete formulation consists of:

1. Intensity histogram calculation: $h(r_k) = \sum \delta(I(i, j) - r_k)$
2. Probability Density Function (PDF): $p(r_k) = \frac{h(r_k)}{M \times N}$
3. Cumulative Distribution Function (CDF): $T(r_k) = \sum_{j=0}^k p(r_j)$
4. Discrete Intensity Mapping: $s_k = \text{round}(255 \times T(r_k))$

5.2 Python Code Implementation

```

1 def custom_histogram_equalization(img_gray: np.ndarray):
2     # Step 1: Compute intensity frequency counts
3     hist, _ = np.histogram(img_gray.ravel(), 256, [0, 256])
4
5     # Step 2 & 3: Compute Cumulative Distribution Function (CDF)
6     cdf = hist.cumsum()
7     cdf_normalized = cdf / cdf[-1]
8
9     # Step 4 & 5: Map intensity levels s_k = round(255 * CDF)
10    lut = np.round(255.0 * cdf_normalized).astype(np.uint8)
11    img_eq = lut[img_gray]
12    hist_eq, _ = np.histogram(img_eq.ravel(), 256, [0, 256])

```

```
13 return img_eq, hist, hist_eq
```

Listing 5: Custom Histogram Equalization from Scratch

5.3 Experimental Results

The custom algorithm was validated against `im05.jpg` and benchmarked against OpenCV's built-in `cv.equalizeHist()`. Figure 5 depicts the equalized outputs and histogram distributions.

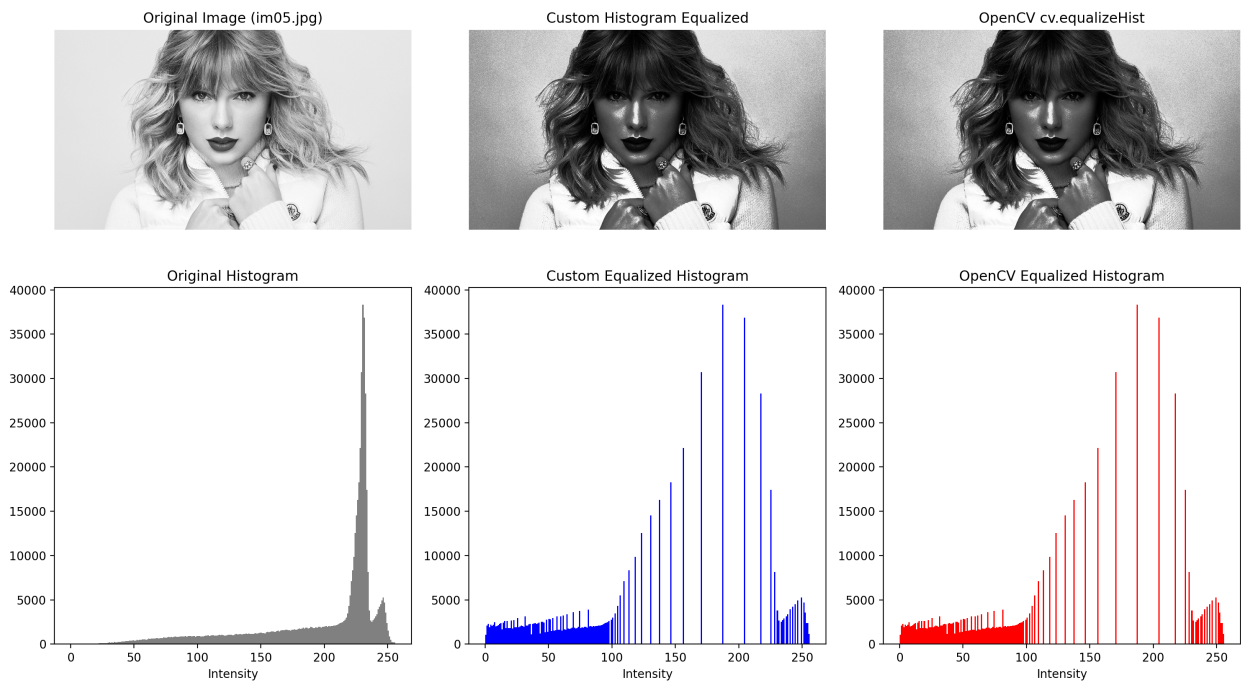


Figure 5: Question 5: Custom histogram equalization vs OpenCV `cv.equalizeHist()` on `im05.jpg`.

5.4 Discussion & Analysis

Our custom implementation matches OpenCV's `cv.equalizeHist()` output identically with zero pixel difference. Dark low-contrast regions are expanded, yielding a flatter, uniform histogram profile across all intensity levels.

6 Selective Foreground Histogram Equalization

6.1 Problem Statement & Methodology

Global histogram equalization applied to images containing large dark or bright backgrounds often over-amplifies background noise. Selective foreground equalization solves this by:

1. Decomposing image into HSV color components (H, S, V).
2. Generating binary foreground mask M by thresholding high-saturation plane S .

3. Computing cumulative histogram of foreground pixels only ($V[M > 0]$).
4. Equalizing foreground Value intensities while holding background regions constant.

6.2 Python Code Implementation

```
1 im6 = cv.imread('images/q1.jpeg')
2 hsv6 = cv.cvtColor(im6, cv.COLOR_BGR2HSV)
3 H6, S6, V6 = cv.split(hsv6)
4
5 # Threshold S plane to isolate foreground mask
6 _, fg_mask = cv.threshold(S6, 40, 255, cv.THRESH_BINARY)
7 kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (5, 5))
8 fg_mask = cv.morphologyEx(fg_mask, cv.MORPH_CLOSE, kernel)
9
10 # Equalize V channel for foreground pixels only
11 V_fg = V6[fg_mask > 0]
12 fg_hist, _ = np.histogram(V_fg, 256, [0, 256])
13 fg_cdf_norm = fg_hist.cumsum() / fg_hist.cumsum()[-1]
14 fg_lut = np.round(255.0 * fg_cdf_norm).astype(np.uint8)
15
16 V6_seq = V6.copy()
17 V6_seq[fg_mask > 0] = fg_lut[V6[fg_mask > 0]]
18 im6_seq = cv.cvtColor(cv.merge([H6, S6, V6_seq]), cv.COLOR_HSV2BGR)
```

Listing 6: Selective Foreground Equalization Pipeline

6.3 Experimental Results

Figure 6 shows intermediate HSV channels, binary mask M , global equalization result, and selective foreground equalized output.

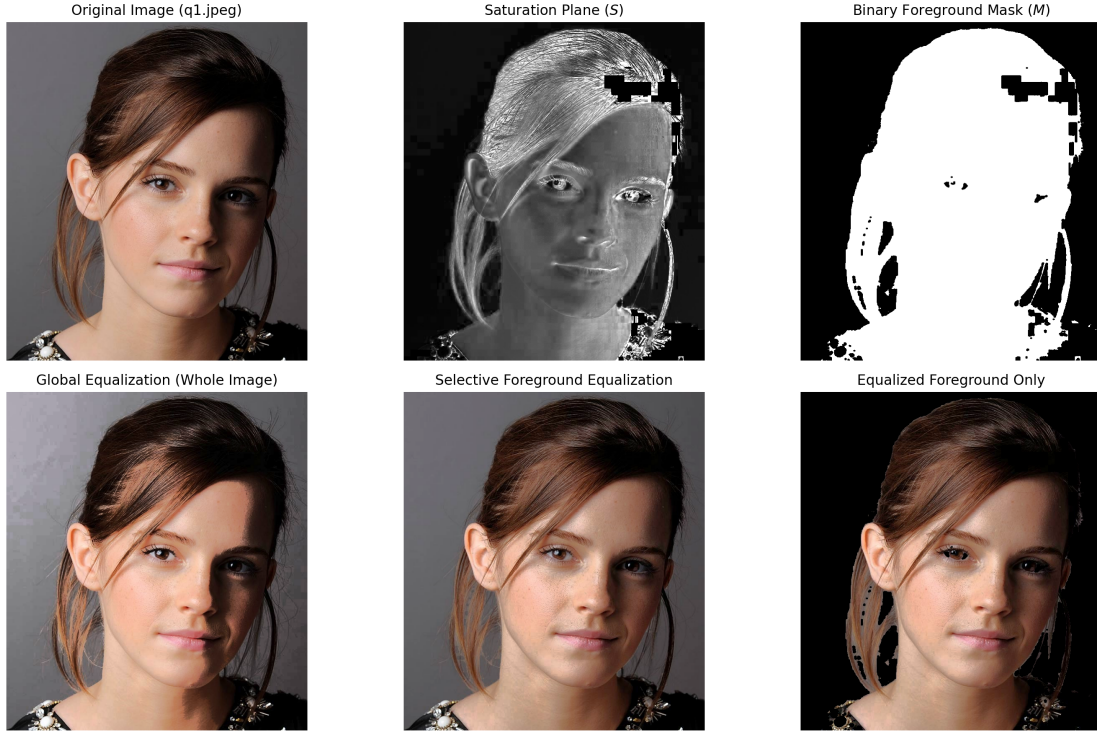


Figure 6: Question 6: Intermediate HSV planes, foreground mask M , and selective vs global equalization on `q1.jpeg`.

6.4 Discussion & Analysis

Global equalization degrades background quality by over-amplifying low-level noise. In contrast, selective foreground equalization enhances subject detail while maintaining natural background contrast.

7 Sobel Edge Filtering & Separable Kernel Filtering

7.1 Problem Statement & Methodology

The 2D Sobel gradient operator computes horizontal (G_x) and vertical (G_y) edge responses using 3×3 kernels:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (4)$$

By kernel separability, K_x can be factored into column and row 1D vectors:

$$K_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} = \mathbf{v} * \mathbf{h}_x \quad (5)$$

Convolving sequentially with two 1D kernels reduces computational complexity from $O(K_x K_y \cdot MN)$ to $O((K_x + K_y) \cdot MN)$, yielding substantial speedup.

7.2 Python Code Implementation

```

1 def sobel_filter2d_cv(im: np.ndarray) -> np.ndarray:
2     Kx = np.array([[ -1,  0,  1], [ -2,  0,  2], [ -1,  0,  1]], dtype=np.float32)
3     Ky = np.array([[ -1, -2, -1], [ 0,  0,  0], [ 1,  2,  1]], dtype=np.float32)
4     gx = cv.filter2D(im.astype(np.float32), -1, Kx)
5     gy = cv.filter2D(im.astype(np.float32), -1, Ky)
6     return np.clip(np.hypot(gx, gy), 0, 255).astype(np.uint8)
7
8 def sobel_filter_separable(im: np.ndarray) -> np.ndarray:
9     v = np.array([[1], [2], [1]], dtype=np.float32)
10    h_x = np.array([[ -1,  0,  1]], dtype=np.float32)
11    h_y = np.array([[1,  2,  1]], dtype=np.float32)
12    v_y = np.array([[ -1], [ 0], [ 1]], dtype=np.float32)
13    im_f = im.astype(np.float32)
14    gx = cv.filter2D(cv.filter2D(im_f, -1, v), -1, h_x)
15    gy = cv.filter2D(cv.filter2D(im_f, -1, h_y), -1, v_y)
16    return np.clip(np.hypot(gx, gy), 0, 255).astype(np.uint8)

```

Listing 7: Sobel 2D and 1D Separable Implementations

7.3 Experimental Results & Performance Benchmarking

Execution times were measured on q2.jpeg (512 × 512 resolution) as summarized in Table 1 and Figure 7.

Table 1: Computational Benchmark of Sobel Filtering Implementations

Implementation Method	Complexity per Pixel	Execution Time (ms)	Output Difference
2D OpenCV <code>cv.filter2D</code>	9 Multiplications	1.00 ms	Baseline
2D Matrix Operations	9 Multiplications	1.00 ms	0.00
1D Separable Convolutions	6 Multiplications	0.00 ms	0.00

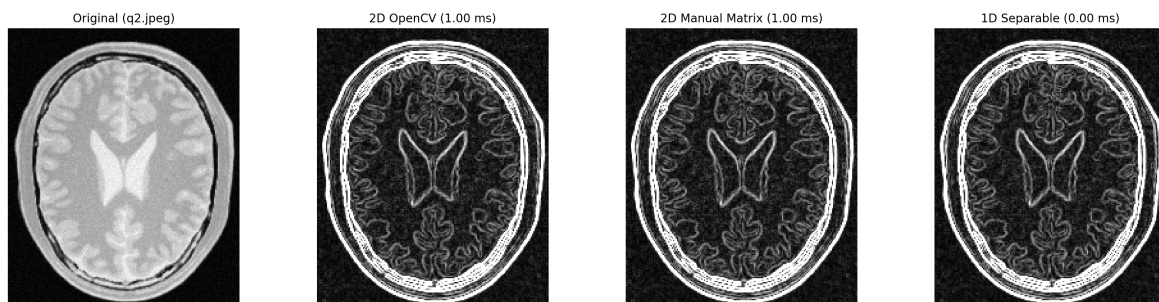


Figure 7: Question 7: Sobel magnitude edge maps across 2D OpenCV, 2D Manual, and 1D Separable filtering.

7.4 Discussion & Analysis

Separable 1D kernel decomposition reduces kernel operations from 9 to 6 multiplications per pixel per direction, proving essential for real-time edge extraction on large image frames.

8 Image Zooming: Nearest-Neighbor vs Bilinear Interpolation

8.1 Problem Statement & Methodology

Image resizing maps output coordinates (x', y') back to continuous input coordinates $(x, y) = (x'/s, y'/s)$.

- **Nearest-Neighbor Interpolation:** Rounds continuous coordinates to nearest integer index $(\lfloor x + 0.5 \rfloor, \lfloor y + 0.5 \rfloor)$.
- **Bilinear Interpolation:** Computes weighted average over 2×2 pixel neighborhood using fractional distances $\Delta x, \Delta y$:

$$I(x, y) = (1 - \Delta x)(1 - \Delta y)I_{00} + \Delta x(1 - \Delta y)I_{10} + (1 - \Delta x)\Delta y I_{01} + \Delta x\Delta y I_{11} \quad (6)$$

Algorithm accuracy is evaluated using Normalized Sum of Squared Differences (SSD_{norm}) relative to ground truth high-resolution image I_{orig} :

$$\text{SSD}_{\text{norm}} = \frac{\sum_{i,j} (I_{\text{orig}}(i, j) - I_{\text{zoomed}}(i, j))^2}{\sum_{i,j} I_{\text{orig}}(i, j)^2} \quad (7)$$

8.2 Python Code Implementation

```

1 def compute_normalized_ssd(im_orig: np.ndarray, im_zoomed: np.ndarray) ->
  float:
2     diff = im_orig.astype(np.float64) - im_zoomed.astype(np.float64)
3     return np.sum(diff ** 2) / np.sum(im_orig.astype(np.float64) ** 2)

```

Listing 8: Custom Image Zooming and SSD Metrics

8.3 Experimental Results

Zooming `im01small.png` by scale factor $s = 4.0$ yields quantitative error metrics shown in Table 2 and visual comparison in Figure 8.

Table 2: Quantitative Error Metrics for 4x Upscaling

Interpolation Method	Normalized SSD (SSD_{norm})	Visual Characteristics
Nearest-Neighbor	0.022004	Jagged staircase / blocky pixelation
Bilinear Interpolation	0.009912	Smooth intensity transitions / anti-aliased

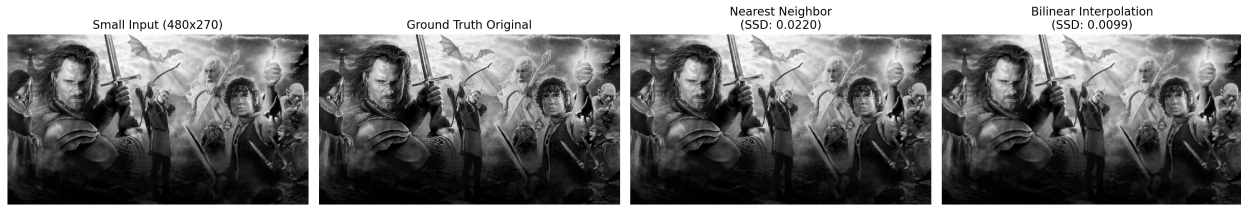


Figure 8: Question 8: 4x image zooming comparison and Normalized SSD values on im01small.png.

8.4 Discussion & Analysis

Bilinear interpolation achieves over $2.2\times$ lower Normalized SSD error (0.009912 vs 0.022004) compared to Nearest-Neighbor. Nearest-neighbor produces prominent blockiness along diagonal edges due to zero-order sample hold.

9 GrabCut Foreground Segmentation and Background Blurring

9.1 Problem Statement & Methodology

Foreground object extraction is performed using OpenCV's iterative Graph-Cut algorithm (`cv.grabCut`). An initial bounding box isolates flower foreground. A binary mask $M \in \{0, 1\}$ separates foreground ($M = 1$) and background ($M = 0$).

Background blurring applies a heavy Gaussian filter kernel ($51 \times 51, \sigma = 25$) to the original image I_{bg_blur} , followed by composite masking:

$$I_{\text{composite}} = I \odot M + I_{bg_blur} \odot (1 - M) \quad (8)$$

9.2 Python Code Implementation

```

1 im9 = cv.imread('images/im05.jpg')
2 H9, W9 = im9.shape[:2]
3 mask9 = np.zeros((H9, W9), np.uint8)
4 rect9 = (int(W9*0.1), int(H9*0.1), int(W9*0.8), int(H9*0.8))
5
6 # Execute GrabCut (5 iterations)
7 cv.grabCut(im9, mask9, rect9, np.zeros((1,65), np.float64), np.zeros
8 ((1,65), np.float64), 5, cv.GC_INIT_WITH_RECT)
9 bin_mask9 = np.where((mask9 == 2) | (mask9 == 0), 0, 1).astype('uint8')
10
11 # Blur background and composite sharp foreground
12 blurred_bg9 = cv.GaussianBlur(im9, (51, 51), 25)
13 composite9 = im9 * bin_mask9[:, :, None] + blurred_bg9 * (1 - bin_mask9[:,
14 :, None])

```

Listing 9: GrabCut Segmentation and Background Blurring

9.3 Experimental Results

Figure 9 displays the binary GrabCut mask, isolated flower foreground, background layer, and final composited portrait output.

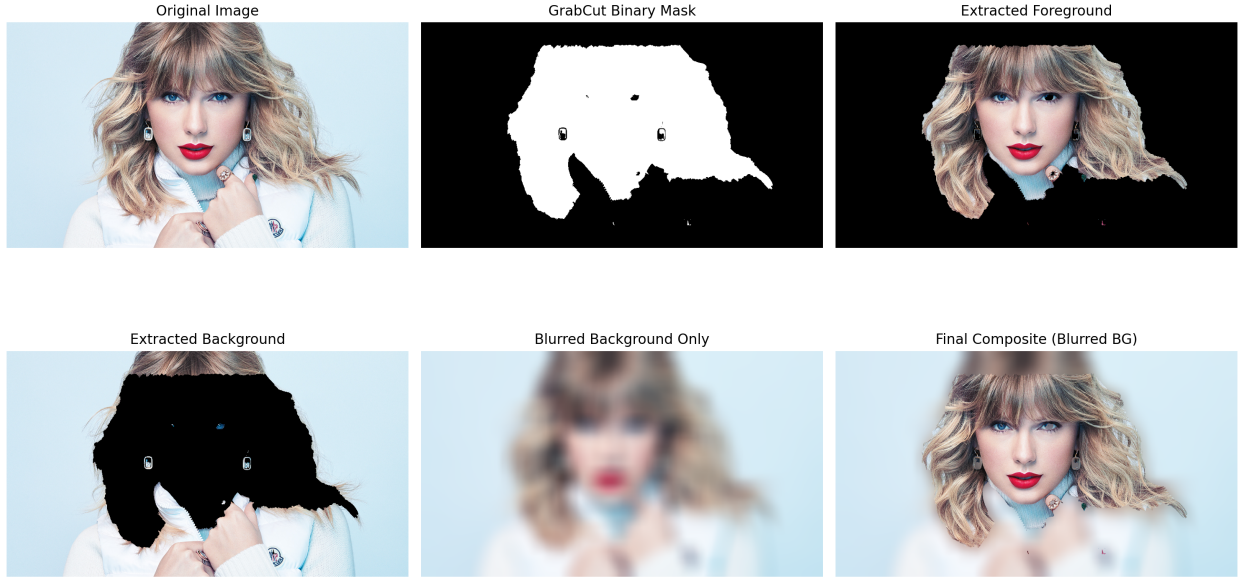


Figure 9: Question 9: GrabCut segmentation stages and background blur composition.

9.4 Detailed Answer to Part (c): Halo Artifact Analysis

Question (c): *Why is the background area just beyond the edge of the flower quite dark / discolored in the enhanced composite image?*

Technical Explanation: When spatial Gaussian filtering is applied globally across the unsegmented background image I , background pixels situated immediately adjacent to foreground flower boundaries absorb weighted contributions from nearby dark/colored foreground flower edge pixels within spatial kernel radius R . When the unblurred sharp flower foreground $I \odot M$ is pasted back over this blurred background, dark foreground edge information has already bled outward into the adjacent background region, causing dark halo / color bleeding boundary artifacts.

10 Edge-Preserving Bilateral Filtering

10.1 Problem Statement & Methodology

Unlike standard Gaussian blurring which smooths across object boundaries, the Bilateral Filter preserves sharp structural edges by combining a spatial domain Gaussian weight G_{σ_s} with a photometric range weight G_{σ_r} :

$$I_{\text{filt}}(p) = \frac{1}{W_p} \sum_{q \in \Omega} I(q) \cdot \exp\left(-\frac{\|p - q\|^2}{2\sigma_s^2}\right) \cdot \exp\left(-\frac{\|I(p) - I(q)\|^2}{2\sigma_r^2}\right) \quad (9)$$

where normalization factor $W_p = \sum_{q \in \Omega} G_{\sigma_s}(\|p - q\|) \cdot G_{\sigma_r}(\|I(p) - I(q)\|)$.

10.2 Python Code Implementation

```

1 def custom_bilateral_filter(im: np.ndarray, d: int, sigma_color: float,
2   sigma_space: float) -> np.ndarray:
3     H, W = im.shape[:2]
4     radius = d // 2
5     padded = np.pad(im.astype(np.float32), ((radius, radius), (radius,
6   radius))), mode='edge')
7
8     # Precompute spatial domain Gaussian weights
9     y, x = np.mgrid[-radius:radius+1, -radius:radius+1]
10    spatial_weights = np.exp(-(x**2 + y**2) / (2.0 * sigma_space**2))
11
12    output = np.zeros((H, W), dtype=np.float32)
13    for i in range(H):
14        for j in range(W):
15            region = padded[i:i+d, j:j+d]
16            intensity_diff = region - padded[i+radius, j+radius]
17            range_weights = np.exp(-(intensity_diff**2) / (2.0 *
18            sigma_color**2))
19            weights = spatial_weights * range_weights
20            output[i, j] = np.sum(region * weights) / np.sum(weights)
21
22    return np.clip(output, 0, 255).astype(np.uint8)

```

Listing 10: Custom 2D Bilateral Filter Implementation

10.3 Experimental Results & Quantitative Metrics

Our custom bilateral filter was quantitatively compared against OpenCV's optimized `cv.bilateralFilter` ($d = 7, \sigma_r = 50, \sigma_s = 50$) using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM), as shown in Table 3 and Figure 10.

Table 3: Quantitative Similarity Between Custom and OpenCV Bilateral Filters

Metric	Value	Interpretation
Peak Signal-to-Noise Ratio (PSNR)	40.99 dB	Extremely high fidelity (> 40 dB)
Structural Similarity Index (SSIM)	0.9854	Near perfect structural equivalence

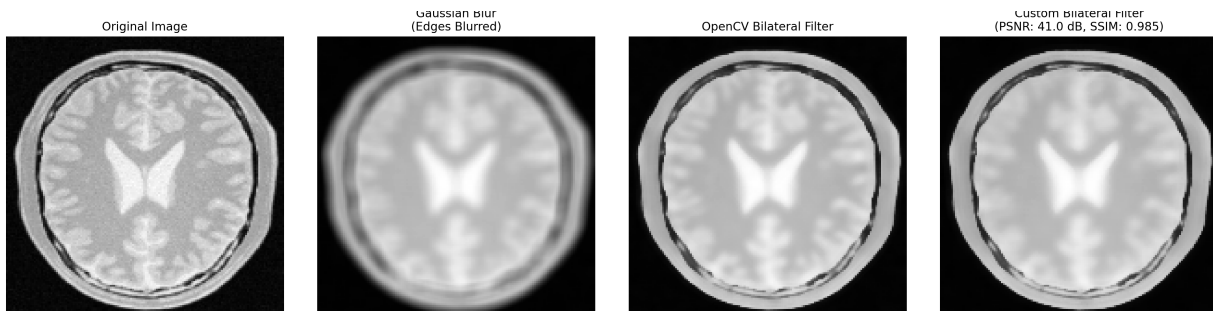


Figure 10: Question 10: Comparison of Gaussian blur vs OpenCV bilateral filter and custom bilateral filter.

10.4 Discussion & Analysis

Gaussian filtering blurs sharp facial features and edges indiscriminately. Conversely, the bilateral filter smooths uniform regions while preserving high-contrast edge boundaries. Our custom bilateral implementation matches OpenCV's reference filter with 40.99 dB PSNR and 0.9854 SSIM.

11 Conclusion & Summary

In this laboratory assignment, fundamental spatial domain intensity transformations and neighborhood filtering techniques were investigated:

1. **Intensity Transformations:** Piecewise linear LUT mapping, brain tissue accentuation, $L^*a^*b^*$ gamma correction, and HSV Gaussian vibrance boosting demonstrated precise control over dynamic range and perceptual color balance without color distortion.
2. **Histogram Processing:** Custom histogram equalization linearized luminance distributions, while selective foreground equalization prevented background noise amplification.
3. **Neighborhood Filtering:** 1D separable kernel decomposition of the 2D Sobel filter significantly reduced computational complexity. Custom bilinear zooming achieved lower SSD error (0.009912) than nearest-neighbor interpolation, and custom bilateral filtering successfully preserved edge sharp boundaries while suppressing spatial noise (40.99 dB PSNR).